

Conditional Information Extraction with Diffusion Model on Fact-Condition Star Graph

Yunxiao Yang
Tongji University
Shanghai, China
yang_98@tongji.edu.cn

Jianting Chen*
Tongji University
Shanghai, China
tj_chenjt@tongji.edu.cn

Xiaoying Gao
Tongji University
Shanghai, China
gxying@tongji.edu.cn

Zaiyuan Di
Tongji University
Shanghai, China
dizaiyuan@tongji.edu.cn

Yang Xiang
Tongji University
Shanghai, China
tjdxxiangyang@gmail.com

Abstract

Conditional Knowledge Graphs (CKGs) extend traditional knowledge graphs by incorporating conditional constraints, enabling a more accurate understanding of complex knowledge with conditional constraints for the semantic web. Conditional information extraction (CIE) aims to extract not only traditional fact triples but also their corresponding conditional qualifiers, forming quintuples that represents these constraints. Existing CIE methods typically treat conditional quintuples as flat structures, overlooking the hierarchical dependencies. Additionally, they often require exploring all possible mention combinations, leading to a large interaction space. These two issues hinder the extraction performance. To this end, we propose a Diffusion Model on Fact-condition Star Graph for CIE (Diff-CIE). We adapt a star graph structure where fact triples serve as central nodes and conditional tuples as leaf nodes, explicitly modeling the hierarchical dependencies. We then leverage the diffusion model to reformulate CIE as a progressive denoising process on these nodes, refining a fixed number of noised nodes into quintuples, thereby reducing the interaction space. Furthermore, to mitigate the inherent optimization instability in traditional diffusion-based information extraction methods, we introduce a deterministic in-order matching strategy to provide an auxiliary constraint. Extensive experiments on three datasets demonstrate that Diff-CIE consistently outperforms state-of-the-art baselines and has higher efficiency, achieving an improvement in F1 metric of over 1.19%, validating the effectiveness of our methods.

CCS Concepts

• Computing methodologies → Information extraction.

Keywords

Conditional Knowledge Graph, Condition Information Extraction, Diffusion Model

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792449>

ACM Reference Format:

Yunxiao Yang, Jianting Chen, Xiaoying Gao, Zaiyuan Di, and Yang Xiang. 2026. Conditional Information Extraction with Diffusion Model on Fact-Condition Star Graph. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792449>

1 Introduction

Knowledge graphs (KGs) are representative implementations of the semantic web, providing structured representations of factual information. However, much real-world knowledge is conditionally constrained, such as time, location, and scope [1–4]. For example, about 30% facts in Freebase are conditionally constrained [5]. Existing KGs typically represent facts as simple triples, overlooking external conditional qualifiers [6–8]. Without these qualifiers, facts may become invalid in the diverse spatiotemporal contexts of the web, leading to ambiguous representations that hinder typical web applications like credibility assessment and fact verification [1, 4, 9, 10]. Therefore, introducing conditions and constructing conditional knowledge graphs (CKGs) that capture fine-grained constraints has become an important direction.

Conditional information extraction (CIE) is the primary approach for CKG construction to enhance the expressiveness of semantic web. Moving beyond traditional information extraction that focuses solely on fact triples, CIE aims to further extract one or more conditional qualifiers for each triple to express constraints and qualifications [2, 11]. These qualifiers consist of condition types and mentions, forming conditional quintuples with fact triples, as shown in Figure 1. For example, the fact triple (*Tom Holland*, *award received*, *Saturn Award*) is constrained by the conditional qualifier (*series ordinal*, *43rd*), resulting in the quintuple (*Tom Holland*, *award received*, *Saturn Award*, *series ordinal*, *43rd*).

Existing CIE methods can be divided into two strategies: pipeline and joint extraction methods. Pipeline methods regard conditional qualifiers as special relations [12, 13], and extract fact triples and condition tuples separately. However, this strategy overlooks the interdependence between fact triples and conditional tuples, limiting its expressive capacity. Joint methods extract fact triples and their associated conditions simultaneously. This strategy often designs specialized paradigms for CIE, such as cube filling [5], cyclic graph extraction [14], which have demonstrated strong performance.

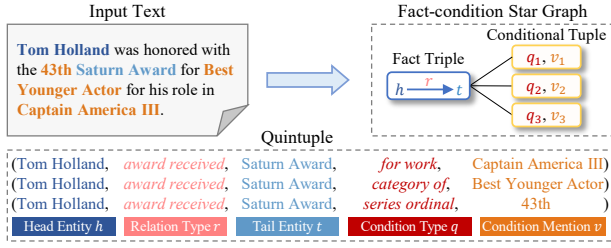


Figure 1: An example for conditional information extraction task, which contains 3 conditional quintuples.

However, existing methods still encounter two challenging problems. **Challenge 1: Dependency Oversight.** Current methods generally treat conditional quintuples as a flat structure, overlooking the potential hierarchical dependencies between fact triples and conditional tuples and thereby reducing the extraction quality. **Challenge 2: Large Interaction Space.** Previous joint extraction methods usually adopt a discriminative paradigm that needs to consider all possible mention combinations among three types of mentions, i.e., head, tail and conditional mentions. For a sentence of length L , the potential interaction space can reach $O(L^3)$. Within such a large space, only a small number of combinations correspond to valid conditional quintuples, thereby impeding model learning, especially given the scale of web data.

To address these challenges, we propose a Diffusion Model on Fact-condition Star Graph for CIE (**Diff-CIE**). For the challenge of dependency oversight, inspired by the triple term and RDF reification in semantic web [15], we construct a fact-condition star graph to explicitly model the hierarchical dependencies. Specifically, each fact triple serves as the central node and leaf nodes represent the conditional tuples constraining the triple. This structure supports matching each fact triple with its corresponding conditions in a hierarchical manner. For the challenge of large interaction space, given the potential of diffusion models for structured prediction such as information extraction [16, 17] and object detection [18], we reformulate CIE as a denoising diffusion process. With set prediction [19, 20], this process gradually denoises and outputs a fixed set of N noised proposal nodes to reconstruct the fact-condition star graph, compressing the interaction space to N ($N \ll L^3$).

Diffusion models for structured prediction typically rely on permuted bipartite matching for optimization [16, 17]. However, its inherent instability, where minor prediction changes may cause an enormous change in the matching outcome, renders ambiguous and inconsistent optimization [20–22]. To address this, we introduce a deterministic in-order matching strategy to provide an auxiliary constraint, which offers stable optimization guidance. In summary, our contributions are as follows:

- We construct a novel fact-condition star graph structure to explicitly model the hierarchical dependencies between fact triples and conditional qualifiers, thereby capturing the complex associations in CIE more accurately.
- We propose Diff-CIE, a diffusion model on star graph for CIE. It reformulates CIE as a denoising diffusion process, and obtains conditional quintuples by gradually refining a set of noised nodes, effectively reducing the interaction space.

- Diff-CIE introduces an auxiliary in-order matching mechanism to provide a stable constraint signal, guiding permutation matching and mitigating its inherent instability.
- Extensive experimental results on three benchmark datasets demonstrate that Diff-CIE outperforms state-of-the-art methods, with over an 1.19% improvement in F1 score. Further ablation studies confirm the effectiveness of each module.

2 Related Work

2.1 Condition Information Extraction

Existing CIE works can be divided into two categories. The first category decomposes conditional structures into basic units and process them in a pipeline manner. Jia et al. [12] decompose relations and conditions with a two-stage architecture. Akimoto et al. [13] regard conditions as special relations within a unified extraction framework. Wang and Yang [23] use an RE model for fact triples, followed by a reading comprehension (MRC) approach for conditions. Ren et al. [24] enhance MRC with continuous prompting. Yuan et al. [25] introduce a reinforcement learning framework to apply traditional RE models to CIE. ReSel [26] retrieves candidates to construct an entity-correlation graph and aggregates results from multiple views. Some works further model correlations between fact triples and conditional tuples. Peng et al. [27] and Song et al. [28] develop graph-LSTMs over dependency graphs to capture multi-granular correlations. EDCRE [29] introduces entity-linking and discourse-relation features for long-distance dependencies. IAN [30] integrates auxiliary Wikidata information with internal text features.

Another category focuses on joint extraction. MIMO [1] designs a multi-input multi-output sequence tagging model for tuples sharing same mentions. Jiang et al. [2] adopt iterative self-training to exploit large-scale unlabeled data. Text2NKG [31] examines fine-grained entity order differences via span-tuple classification approach with hetero-ordered merging. MacroIE [32] designs a table-filling method based on token span graphs, and CubeRE [5] extends the two-dimensional table into three-dimensional cubes and employs a pruning strategy to mitigate negative class imbalance. Some approaches model CKGs as graphs: Lv et al. [11] regard relations and conditions as special nodes in hierarchical graphs, Mirror [14] proposes a multi-span cyclic graph strategy to infer quintuples, and LDNet [33] incorporates relation and condition types into the input with a label-drop mechanism to reduce noisy relation impacts.

Some other works address specific challenges, employing distant supervision [34] and active learning [35] for limited high-quality data, or timeline decomposition [36] for temporal conditions.

2.2 Diffusion Model in Language

Diffusion models [37, 38] are deep generative models originally developed for image generation. Although they have shown strong performance in image and audio domains, they primarily operate on continuous data. Due to the discrete nature of text, applying diffusion models in NLP remains challenging. Some studies have integrated diffusion models with text via embedding-based representations [39] or absorbing multinomial diffusion processes [40]. Beyond these foundational efforts, diffusion models have been extended to downstream NLP tasks such as conditional Seq2Seq generation [41] and text summarization [42, 43].

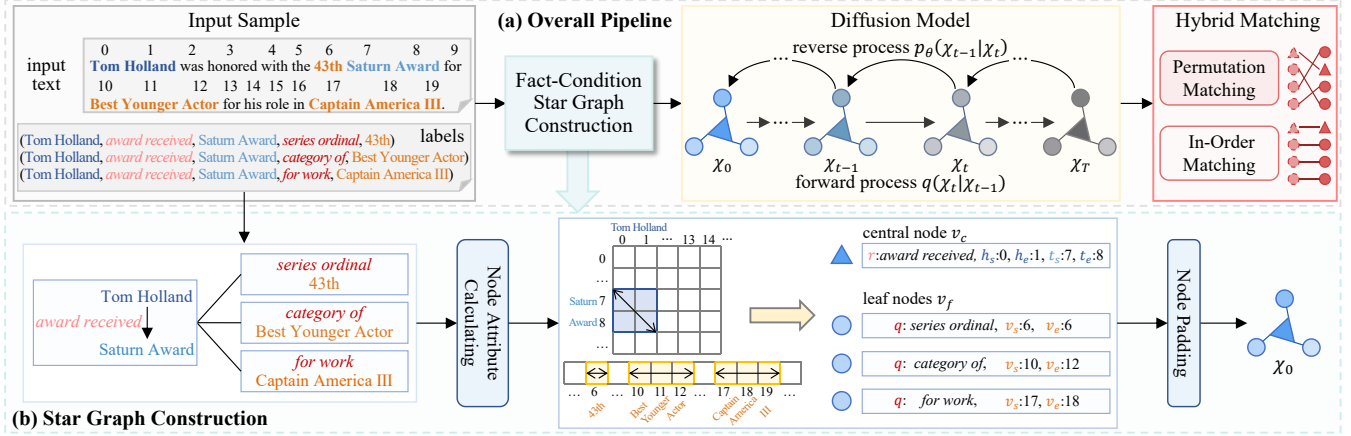


Figure 2: The overview of our proposed Diff-CIE. (a) The overall pipeline, where q is the forward process and p_θ is the reverse process. (b) The detailed process of fact-condition star graph construction in Diff-CIE.

In information extraction, previous works have primarily focused on entity relation extraction [16, 17] and event extraction [44]. PGAD [44] uses diffusion models to generate diverse, context-aware prompts to better model interactions among different argument roles. DiffusionNER [16] and IPED [17] reformulate extraction as a denoising process over span or block boundaries, progressively refining the boundaries to obtain extraction results, and further adopt set prediction with optimal bipartite matching for optimization.

3 Preliminary

3.1 Diffusion Model

Forward process. The original input sample x_0 is corrupted by progressively adding Gaussian noises. The forward transition $x_{t-1} \rightarrow x_t$ can be formulated as:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t \mathbf{I}) = \sqrt{1 - \beta_t}x_0 + \sqrt{\beta_t}\epsilon, \quad (1)$$

where $q(x_t | x_{t-1})$ denotes the forward noising process from x_{t-1} to x_t , $t = 1, 2, \dots, T$ represents the diffusion steps, $\mathcal{N}$ denotes the Gaussian distribution, β_t controls the noise scale, $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ is random Gaussian noise. As t increases, the noisy data x_T will gradually converge to complete Gaussian noise [37]. The forward process of the diffusion model can be regarded as a Markov chain, so the noise injection process can be expressed as:

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t)\mathbf{I}), \quad (2)$$

where $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{t'=1}^t \alpha_{t'}$, x_0 and x_t denote the original and noised data at timestep t , respectively.

Reverse process. Starting from $x_T \sim \mathcal{N}(0, \mathbf{I})$, the model learns a parametric distribution $p_\theta(x_{t-1} | x_t)$ to remove noise from x_t and recover x_{t-1} , thereby progressively reconstructing the original sample. The transition $x_t \rightarrow x_{t-1}$ is defined as:

$$p_\theta(x_{t-1} | x_t) \sim \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)), \quad (3)$$

where μ_θ and Σ_θ are the mean and variance of the predicted noise distribution. μ_θ is reparameterized through the denoising network

$f_\theta(\cdot)$:

$$\mu_\theta(x_t, t) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t}f_\theta(x_t, t), \quad (4)$$

where $f_\theta(x_t, t)$ predicts the original sample x_0 conditioned on the noisy sample x_t and timestep t . Σ_θ is typically set as a constant variance in previous work [16, 37, 45].

3.2 Problem Statement

Given an input sentence of L words $s = \{w_1, w_2, \dots, w_L\}$, the objective of the condition information extraction is to extract a set of conditional facts $\mathcal{F} = \{\tau_1, \tau_2, \dots, \tau_M\}$.

Formally, a conditional fact $\tau \in \mathcal{F}$ is represented as a quintuple: $\tau = (h, r, t, q, v)$. The entity mentions h, t and condition mention v are a consecutive span of words in s . $r \in \mathcal{R}$ is the relation between h and t , where $\mathcal{R}$ is the predefined relation set. $q \in \mathcal{Q}$ is the condition type corresponding to v , where $\mathcal{Q}$ is the condition set.

4 Methodology

In this section, we introduce Diff-CIE, a diffusion-based CIE model, as shown in Figure 2. We construct a fact-condition star graph structure for conditional quintuples, and introduce a denoising diffusion model to reconstruct the fact-condition graph from noised inputs. In addition, we propose a hybrid matching strategy that introduces in-order matching to provide auxiliary supervision.

4.1 Fact-Condition Star Graph Construction

To more effectively model the dependencies between conditional qualifiers and fact triples, we convert the original quintuple into a star graph format, as shown in Figure 2(b). Specifically, we decompose each τ into two parts: the fact triple $\tau_f = (h, r, t)$, representing the core fact, and the conditional tuple $\tau_c = (q, v)$, representing the corresponding conditional qualifier. All conditional tuples sharing the same fact triple are then grouped to form a star graph, where τ_f serves as the central node (i.e., fact node) and its associated leaf node set (i.e., condition nodes). $\Phi(\tau_f)$ can be expressed as:

$$\Phi(\tau_f) = \{(q_i, v_i) \mid (h, r, t, q_i, v_i) \in \mathcal{F}, \tau_f = (h, r, t)\}. \quad (5)$$

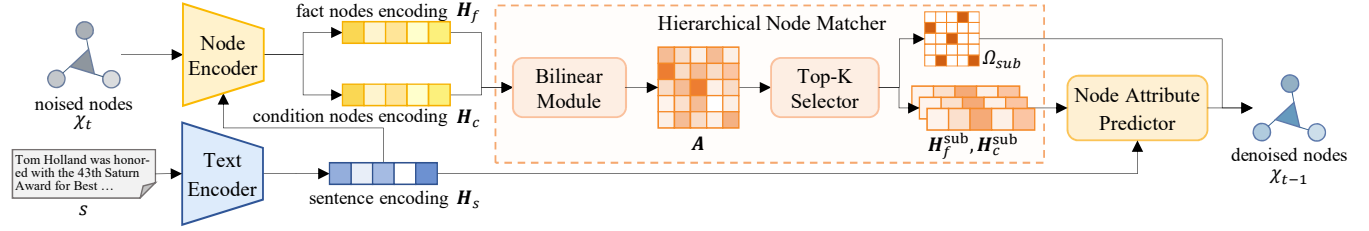


Figure 3: The detailed architecture of the denoising network in Diff-CIE.

Thus, the entire quintuples can be converted into a hierarchical star graph structure, where all edges are directed from the fact node to the condition node:

$$\mathcal{G} = \{\tau_f \mapsto \Phi(\tau_f)\}. \quad (6)$$

Our goal is to extract the corresponding fact-condition star graph $\mathcal{G}$ from the input text s .

4.1.1 Node Attribute Calculating. To satisfy the input requirements of the diffusion model, we convert the attributes of fact and condition nodes in $\mathcal{G}$ into a unified representation $(\omega, \{\eta^\kappa\})$ including the relation or condition type ω and span indices $\{\eta^\kappa\}$:

$$\begin{aligned} \mathcal{V}_f &= \{\psi(\tau_f)_i\}_{i=1}^{M_1}, \quad \psi(\tau_f) = (\omega_f, \{\eta_f^\kappa\}) = (r, \{h_s, h_e, t_s, t_e\}), \\ \mathcal{V}_c &= \{\psi(\tau_c)_i\}_{i=1}^{M_2}, \quad \psi(\tau_c) = (\omega_c, \{\eta_c^\kappa\}) = (q, \{v_s, v_e\}), \end{aligned} \quad (7)$$

where ψ is the convert operation, M_1, M_2 denote the total number of fact nodes and condition nodes, respectively. $*, *_e$ indicate the start and end span indices of the mention. We then normalize these indices based on sentence length.

4.1.2 Node Padding. In practice, the number of quintuples varies across sentences, and specific facts may be associated with varying numbers of conditions. Therefore, we utilize padding $(\cdot)$ to add some additional nodes to the original ground truth so that all samples contain a fixed number of proposal nodes N_1, N_2 . This strategy enables the model to handle facts with multiple conditions uniformly within a fixed interaction space. Following previous work [16, 17], these additional nodes are obtained by using random noise as null nodes. Finally, the initial input to the diffusion model is expressed as:

$$\chi_0 = (\chi_0^f, \chi_0^c) = (\text{padding}(\mathcal{V}_f; N_1), \text{padding}(\mathcal{V}_c; N_2)). \quad (8)$$

4.2 Diffusion Model on Fact-Condition Graph

We propose to reformulate CIE as a denoising diffusion process on noisy graph nodes, as shown in Figure 2(a). It consists of two processes: the forward diffusion process and the reverse denoising process. Under aforementioned graph node indexing format, the forward process progressively add noise to the span indices $*, *_e$ of nodes. The reverse process recovers the clean indices.

4.2.1 Forward Process. Gaussian noise is gradually added into the span indices of the input sample χ_0 . Due to the Markov property of the forward process, given χ_0, χ_t can be reparameterized as a single-step noise addition process:

$$\chi_t = \sqrt{\alpha_t} \chi_0 + \sqrt{1 - \alpha_t} \epsilon, \quad (9)$$

where t is a randomly selected timestep, and $\epsilon \sim \mathcal{N}(0, I)$ is noise sampled from a standard Gaussian distribution.

4.2.2 Reverse Process. Once the noised sample is obtained, the reverse process feeds χ_t into the denoising network $f_\theta(\chi_t, t, s)$. According to $p_\theta(\chi_{t-1} | \chi_t, s)$, it reconstructs $\hat{\chi}_0$ conditioned on the input sentence s . We describe the detailed architecture of $f_\theta(\cdot)$ in Section 4.3.

We adopt DDIM [38] to accelerate sampling. We define z as an increasing sub-sequence of $[1, 2, \dots, T]$ of length l , where $z_l = T$. The denoising process from z_l to z_{l-1} can then be expressed as:

$$\begin{aligned} \hat{\chi}_0 &= f_\theta(\chi_{z_l}, z_l, s), \\ \hat{\epsilon}_{z_l} &= \frac{\chi_{z_l} - \sqrt{\alpha_{z_l}} \hat{\chi}_0}{\sqrt{1 - \alpha_{z_l}}}, \\ \hat{\chi}_{z_{l-1}} &= \sqrt{\alpha_{z_{l-1}}} \hat{\chi}_0 + \sqrt{1 - \alpha_{z_{l-1}}} \hat{\epsilon}_{z_l}, \end{aligned} \quad (10)$$

where $\hat{\chi}_0$ and $\hat{\epsilon}_{z_l}$ represent the predicted result and noise recovered from the noisy distribution at timestep z_l . After l iterations of DDIM, the noisy indices are gradually refined.

4.3 Denoising Network Architecture

As shown in Figure 3, given noisy mention indices at timestep z_l and sentence s , the denoising network $f_\theta(\chi_{z_l}, z_l, s)$ gradually generates denoised predictions $\hat{\chi}_0$.

4.3.1 Encoder. Given the sentence $s = \{w_1, w_2, \dots, w_L\}$, we input it into a pre-trained language model and obtain the last hidden layer $\mathbf{H}_s = \{\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_L\} \in \mathbb{R}^{L \times d}$ as the sentence embedding, where L is the sentence length. Based on the span indices of the mentions indicated by the noised graph nodes χ_t^f, χ_t^c , we use the sentence embedding $\mathbf{H}_s$ to calculate the hidden state for each mention span $\tilde{\mathbf{H}}_f \in \mathbb{R}^{N_1 \times d}, \tilde{\mathbf{H}}_c \in \mathbb{R}^{N_2 \times d}$.

To better capture the correlation between node features $\tilde{\mathbf{H}}_f, \tilde{\mathbf{H}}_c$ and the text information $\mathbf{H}_s$, we further utilize attention to integrate context information into the node representation, thus enabling an implicit intervention at the feature level. Specifically, we treat the node embedding as the query and the sentence embedding as the key and value. Take the fact node as an example:

$$\mathbf{H}_f = \text{Attn}(\tilde{\mathbf{H}}_f, \mathbf{H}_s, \mathbf{H}_s), \quad \mathbf{H}_c = \text{Attn}(\tilde{\mathbf{H}}_c, \mathbf{H}_s, \mathbf{H}_s). \quad (11)$$

4.3.2 Hierarchical Node Matcher. This module aims to reconstruct the edges of the star graph. Supposing there are N_1 fact nodes and N_2 condition nodes, this forms $N_1 \times N_2$ fact-condition pairs Ω . The core of node matcher is to evaluate the relative importance of each fact-condition pair and perform pruning based on the importance

matrix to select the top- K subset Ω_{sub} . We use a bilinear module to calculate the importance matrix $\mathbf{A} \in \mathbb{R}^{N_1 \times N_2}$ between $\mathbf{H}_f$ and $\mathbf{H}_c$:

$$\mathbf{A} = \mathbf{H}_f \mathbf{W}_m \mathbf{H}_c^\top, \quad (12)$$

where $\mathbf{W}_m \in \mathbb{R}^{d \times d}$ is learnable weights. The top- K entries with the highest scores are selected as:

$$\Omega_{sub} = \underset{K}{\text{Top-Select}}(\Omega; \mathbf{A}), \quad (13)$$

where $\forall (i, j) \in \Omega_{sub}$ and $(i', j') \in \Omega \setminus \Omega_{sub}$, it holds $\mathbf{A}_{ij} > \mathbf{A}_{i'j'}$. Finally, the selected indices forms the edges of the star graph and candidate node representations $\mathbf{H}_f^{\text{sub}}, \mathbf{H}_c^{\text{sub}}$ are filtered from $\mathbf{H}_f$ and $\mathbf{H}_c$ based on these indices.

4.3.3 Node Attribute Predictor. After acquiring the edges and filtered node representations, we need to further predict the attributes of these nodes. For mention indices, we first calculate the word-level fusion representation of the node and text representations $\mathbf{H}_{fs}, \mathbf{H}_{cs} \in \mathbb{R}^{K \times L \times d}$. Then, we use binary classification span pointers to predict the mention span indices. Specifically, each fact node needs 4 pointers, whereas 2 for the condition node.

$$\begin{aligned} \mathbf{H}_{fs} &= \mathbf{W}_f^{\text{span}} \left[\mathbf{H}_f^{\text{sub}} \oplus \mathbf{H}_s \right], \quad \mathbf{p}_f^\kappa = \sigma \left(\mathbf{W}_f^\kappa \mathbf{H}_{fs} + b_f^\kappa \right) \\ \mathbf{H}_{cs} &= \mathbf{W}_c^{\text{span}} \left[\mathbf{H}_c^{\text{sub}} \oplus \mathbf{H}_s \right], \quad \mathbf{p}_c^\kappa = \sigma \left(\mathbf{W}_c^\kappa \mathbf{H}_{cs} + b_c^\kappa \right) \end{aligned} \quad (14)$$

where $\oplus$ is the broadcast concatenation operation: $\mathbf{F} = \mathbf{X}_1 \oplus \mathbf{X}_2 \Leftrightarrow \mathbf{F}_{i,j,k} = [\mathbf{X}_1]_{i,k} \parallel [\mathbf{X}_2]_{j,k}$. $\mathbf{W}_f^{\text{span}}, \mathbf{W}_c^{\text{span}} \in \mathbb{R}^{2d \times d}$ and $\mathbf{W}_f^\kappa, \mathbf{W}_c^\kappa \in \mathbb{R}^{d \times 1}$ are learnable parameters, $\kappa \in \{*, e\}$ is the start and end span pointer identifier, σ is the sigmoid function. For node types, we construct two classifiers to identify the type of each node.

$$\mathbf{p}_f^{\text{cls}} = \text{softmax} \left(\mathbf{W}_f^{\text{cls}} \mathbf{H}_f^{\text{sub}} + b_f^{\text{cls}} \right), \mathbf{p}_c^{\text{cls}} = \text{softmax} \left(\mathbf{W}_c^{\text{cls}} \mathbf{H}_c^{\text{sub}} + b_c^{\text{cls}} \right), \quad (15)$$

where $\mathbf{W}_f^{\text{cls}} \in \mathbb{R}^{d \times |\mathcal{R}|}, \mathbf{W}_c^{\text{cls}} \in \mathbb{R}^{d \times |\mathcal{Q}|}$ are learnable parameters. Since each span pointer is independent, the fact and condition node results can be calculated simultaneously. Finally, we filter out results with the sum of prediction probabilities less than a threshold γ .

4.4 Training Objective with Hybrid Matching

The optimization objective is to score the predicted node types and span indices and align them with the ground truth in a set-matching manner. For convenience, we use i to uniformly indicate the indices of fact and condition nodes. Given a noised sample χ_t , the denoising network $f_\theta(\cdot)$ produces the prediction set $\hat{\mathbf{Y}}^t = \{\hat{y}_i^t\}_{i=1}^K$, where $\hat{y}_i^t = (\mathbf{p}_i^{\text{cls}}, \{\mathbf{p}_i^\kappa\})$. During training, we first define a scoring function to measure the match degree between predictions and ground truth. This scoring function needs to consider the similarity of both node types and span indices of mentions. It is calculated as:

$$\ell_{\text{match}}(y_i, \hat{y}_j^t) = - \left(\log \mathbf{p}_j^{\text{cls}}[\omega_i] + \sum_{\kappa} \log \mathbf{p}_j^\kappa[\eta_i^\kappa] \right), \quad (16)$$

where ω_i, η_i^κ denote the type and span indices of the i -th ground truth node, $\mathbf{p}_j^*[\cdot]$ is the predicted probability of the j -th node.

Permutation matching is then employed for model optimization. This step aims to find the optimal bipartite matching between predicted nodes and ground truth, and then optimize the loss. Following previous work [16, 19], we adopt the Hungarian algorithm

to obtain the optimal matching. We search for the permutation with the lowest pair wise matching cost to find the optimal bipartite matching between $\mathbf{Y}$ and $\hat{\mathbf{Y}}^t$:

$$\hat{\sigma} = \arg \min_{\sigma \in \mathfrak{S}} \sum_i \ell_{\text{match}}(y_i, \hat{y}_{\sigma(i)}^t). \quad (17)$$

We apply the Hungarian algorithm separately to fact nodes and condition nodes to obtain the overall permutation matching loss:

$$\mathcal{L}_{\text{permutation}} = \frac{1}{K} \sum_{i=1}^K \ell_{\text{match}}(y_i, \hat{y}_{\sigma(i)}^t), \quad (18)$$

where ℓ_{match}^f and ℓ_{match}^c denote the matching costs for the fact and condition nodes, $\hat{\sigma}$ is the optimal assignment obtained by Eq. (17).

However, the instability of permutation matching negatively affects model training. Therefore, we further design a hybrid matching strategy that incorporates **in-order matching** to provide an auxiliary loss for training guidance. Specifically, we obtain $\hat{\mathbf{Y}}^1$ from χ_1 , a lightly noised sample with only one diffusion step, to simulate the case where the input sample is only slightly perturbed. Since $\hat{\mathbf{Y}}^1$ contains only minor noise, it can be considered to retain relatively sufficient information about original labels. Therefore, we adopt in-order matching to align each predicted node with its ground-truth node by position, thereby emphasizing the one-to-one correspondence. The optimization objective for this part is formally expressed as:

$$\mathcal{L}_{\text{in-order}} = \frac{1}{K} \sum_{i=1}^K \ell_{\text{match}}^f(y_i, \hat{y}_i^1). \quad (19)$$

Joint training objective is a weighted combination of these two matching losses. The training and inference algorithms are presented in Appendix A.

$$\mathcal{L} = \mathcal{L}_{\text{permutation}} + \lambda \mathcal{L}_{\text{in-order}}. \quad (20)$$

5 Experiment

In this section, we conduct comprehensive experiments to evaluate our method and answer the following research questions: **(RQ1)** How does Diff-CIE perform compared to the state-of-the-art models in CIE task on extraction settings **(RQ1.1)** and classification settings **(RQ1.2)**? **(RQ2)** How does each component contribute to the overall performance? **(RQ3)** How does Diff-CIE perform in terms of training and inference efficiency? **(RQ4)** How do hyper-parameters influence the effectiveness and efficiency of Diff-CIE? **(RQ5)** How does Diff-CIE perform on fine-grained prediction results?

5.1 Experimental Settings

5.1.1 Datasets. Following previous work [31, 36], we evaluate our method on three widely used CIE datasets: HyperRED [5], Hyper-Temporal [36], and ComplexTRED [36]. Detailed description can be found in Appendix B.

5.1.2 Baselines. We compare our Diff-CIE with the following 12 baselines from four categories: **LLM-based methods**. This category includes both training-free methods (GPT-3.5-Turbo, GPT-4o, Deepseek-v3) and LoRA fine-tuned methods (GoLLIE [46]). **Universal IE methods**. UIE [47] and InstructUIE [48] can be applied to various information extraction tasks by constructing a unified task

Table 1: The overall performance comparison on three datasets.

Model	HyperRED			Hyper-Temporal			ComplexTRED		
	P	R	F1	P	R	F1	P	R	F1
<i>w/o TSD</i>									
GPT3.5-Tubro	12.50	10.18	11.22	15.01	14.68	14.84	14.81	11.06	12.66
GPT-4o	16.89	13.82	15.20	34.02	28.01	30.72	41.20	19.10	26.10
Deepseek-v3	14.60	33.26	20.29	21.39	31.20	25.38	27.67	26.14	26.88
GOLLIE [46]	63.57	55.25	59.12	59.53	50.94	54.90	43.91	33.77	38.18
UIE [47]	68.45	58.97	63.36	65.38	56.80	60.79	44.83	32.05	37.38
InstructUIE [48]	66.29	56.62	61.08	63.92	54.83	59.04	46.50	32.23	38.07
DiffusionNER [16]	71.03	61.40	65.87	63.87	62.87	63.38	46.73	26.93	34.17
IPED [17]	<u>72.08</u>	62.52	66.96	66.37	<u>64.78</u>	<u>65.56</u>	48.19	28.24	35.57
CubeRE [5]	66.39	<u>67.12</u>	66.75	55.31	49.64	52.33	41.69	27.92	33.44
Mirror [14]	71.29	62.46	66.58	72.12	57.67	64.09	<u>48.20</u>	29.00	36.21
LDNet [33]	69.39	66.56	<u>67.95</u>	68.66	59.10	63.52	45.52	31.17	37.00
TSDRE [36]	67.76	66.51	67.13	66.64	61.06	63.73	47.64	<u>34.86</u>	<u>40.26</u>
Diff-CIE (Ours)	74.29	68.47	71.26	<u>69.50</u>	65.73	67.57	50.73	35.04	41.45
<i>w/ TSD</i>									
TSDRE [36]	/	/	/	68.61	64.91	66.71	48.99	37.61	42.55
Diff-CIE (Ours)	/	/	/	68.15	70.96	69.53	52.63	36.33	42.99

The best and second-best results are highlighted in **bold** and underlined respectively. *w/o* or *w/ TSD* refers to whether the results are obtained using the time decomposition trick proposed in TSDRE [36].

schema. **Diffusion IE methods.** DiffusionNER [16] and IPED [17] use the diffusion model for NER and RE, respectively. We expand and apply them to the CIE task. **Advanced CIE methods.** This category includes discriminative models CubeRE [5], Mirror [14], LDNet [33], and generative model TSDRE [36].

5.2 Main Results (RQ1.1)

We compare the performance of our method with baseline methods on three datasets, as shown in Table 1. Our Diff-CIE achieves consistent improvements across all datasets. In particular, it significantly outperforms previous methods on HyperRED and Hyper-Temporal, and also demonstrates clear improvements over previous discriminative methods on ComplexTRED. Specifically, compared to the previous best method, our method improves Precision (P), Recall (R), and F1 on HyperRED by 2.21%, 1.35%, and 3.31%; while improves F1 on Hyper-Temporal and ComplexTRED by 2.01% and 1.19%.

Across different datasets, our Diff-CIE shows more significant improvements on HyperRED. HyperRED is a more complex dataset, containing more diverse conditional qualifiers rather than just temporal conditions. This indicates our method’s superiority in diverse and complex scenarios. We attribute this to the integration of star graph modeling and diffusion mechanism, which better captures the dependencies between conditional qualifiers and fact triples.

Compared to advanced CIE methods, our method achieves superior performance. This advantage likely results from reformulating CIE task as a denoising diffusion process, which reduces the interaction space and task difficulty. Moreover, our method consistently outperforms existing diffusion-based IE methods. This stems from two aspects. First, our star graph modeling naturally emphasizes the

hierarchical dependencies of conditional qualifiers compared to the flat structure. Second, the guidance of in-order matching provides a stable supervisory signal, thereby enhancing model stability.

Compared to universal IE methods, models specifically designed for CIE usually perform better. This reveals the characteristic structural complexity of CIE, thus demonstrating the necessity and effectiveness of specialized designs for CIE. Furthermore, LLM-based methods do not perform satisfactorily, especially training-free methods, which significantly lag behind other methods. Even compared to the LoRA fine-tuned GoLLIE, our Diff-CIE still achieves F1 improvements of 12.14%, 12.67%, and 3.27% on the three datasets. This suggests that relying solely on the general capabilities of LLMs is insufficient for handling the fine-grained conditional constraints involved in CIE and often leads to incomplete quintuple predictions.

Notably, the timeline decomposition trick in TSDRE [36] has limited improvements on our method, likely because the same mention could appear in multiple clauses after decomposition, causing confusion for our discriminative model and weaken the effect of this trick. However, this issue does not affect the generative TSDRE.

5.3 Condition Classification Results (RQ1.2)

We follow the setting in Text2NKG [31] and further evaluate a relatively simplified classification task. In this setting, both the entity mention and condition mention are assumed to be provided along with the input sentence. The model only needs to predict the corresponding relation and condition type, simplifying the information extraction task into a classification task. We choose HyperRED to evaluate model performance as it contains 44 condition types, which is more diverse than other datasets.

Table 2: Performance comparison of condition classification setting on HyperRED.

Model	HyperRED-CLS		
	P	R	F1
GPT-3.5-Tubro	16.08	13.55	14.71
GPT-4o	23.40	19.74	21.42
Deepseek-v3	19.34	42.66	26.61
Text2Event [49]	73.94	70.56	72.21
UIE [47]	76.51	73.02	74.72
InstructUIE [48]	80.88	70.05	75.08
LasUIE [50]	79.62	78.04	78.82
Text2NKG [31]	83.53	86.59	85.03
Diff-CIE (Ours)	89.91	<u>84.37</u>	87.05

As shown in Table 2, our Diff-CIE still demonstrates consistent superiority, achieving an F1 score of 87.05%. Compared to the second-best Text2NKG, our method improves by 2.02%.

Compared to the traditional setting in Section 5.2, all models show significant performance improvements, indicating that locating conditional qualifiers is the primary bottleneck in the task. In comparison, the performance gain for the training-free methods is not as substantial as that for the fine-tuning methods. This may be because the core challenge for LLMs lies in fine-grained type classification and the generation of valid quintuple structures, while merely providing accurate mentions yields limited benefits.

Notably, under the condition classification setting, the performance gap between our Diff-CIE and other models is somewhat reduced. This may be because, when all mentions are given, the complex interactions between conditions and fact triples are simplified, thus reducing the task difficulty. This also reflects that the advantage of our proposed star graph modeling and diffusion mechanism are more evident in complex scenarios. This advantage is partially weakened under the simplified setting.

5.4 Ablation Study (RQ2)

To analyze the contribution of each component in our Diff-CIE to overall performance, we remove the main components to validate their individual impact. The experimental results are shown in Table 3. The results reveal that removing any of these components leads to a relative performance drop.

To validate the effectiveness of our star graph modeling, **w/o star graph** removes the process of reconstructing fact-condition

Table 3: Ablation studies on Hyper-Temporal and ComplexTRED.

Model	Hyper-Temporal			ComplexTRED		
	P	R	F1	P	R	F1
Diff-CIE	69.50	65.73	67.57	50.73	35.04	41.45
w/o star graph	63.35	62.54	63.92	50.05	29.95	37.48
w/o attention	69.36	61.86	65.40	49.91	33.32	39.96
w/o bilinear	68.15	65.80	66.96	48.35	35.50	40.94
w/o in-order matching	69.24	63.06	66.01	53.28	32.79	40.60

Table 4: Performance comparison of model efficiency on Hyper-Temporal.

Model	GPU usage (MB)	Training Time (s)	Infer. Time (ms)
CubeRE	17,208	155	5.90
Mirror	13,442	252	4.66
LDNet	13,328	255	4.76
Diff-CIE	21,430	188	4.03

star graph from node pairs and directly predict the entire quintuple. It degenerates the star graph structure into a flat one. This results in a significant performance decline, with F1 score dropping by 3.65% and 3.97%, respectively. This result highlights the importance of star graph modeling, as it effectively captures the complex hierarchical interactions between fact triples and conditional tuples, thus enhancing the model’s understanding of conditional knowledge.

We also examine the role of attention in the node encoder. When removing this module (**w/o attention**), the model’s performance decreases by 2.17% and 1.49%. In particular, recall decreases significantly by 3.87% on Hyper-Temporal. These results highlight the effectiveness of this module, indicating that it improves node representations, thereby reducing the omission of latent conditional instances. We further evaluate the importance scoring method in Hierarchical Node Matcher (**w/o bilinear**). When replacing the bilinear module with simple dot-product similarity, the performance also slightly decreases, validating the rationale of bilinear modeling.

To evaluate the impact of hybrid matching, **w/o in-order matching** only uses traditional permutation matching, resulting in F1 drops of 1.56% and 0.85%, demonstrating that in-order matching improves training stability by providing a clear supervisory signal.

5.5 Model Efficiency Analysis (RQ3)

We evaluate computational efficiency and complexity through memory usage, training and inference speed. Table 4 shows the comparison between Diff-CIE and 3 baseline models.

Although Diff-CIE inevitably increases memory usage due to the need to maintain the hierarchical fact-condition star graph structure, this overhead is acceptable given its superior extraction performance. Meanwhile, our training efficiency is moderate (188s), significantly outperforming advanced models such as Mirror (252s) and LDNet (255s). Additionally, our method performs best in inference efficiency, with an average cost of 4.03ms per sample. This is attributed to our graph node denoising and the set prediction task format, which directly outputs the final quintuple set. This approach avoids the need to exhaustively traverse all mention combinations, thus reducing the computational burden. Therefore, our method does not introduce excessive training time costs despite the introduction of complex modeling mechanisms.

5.6 Parameter Analysis (RQ4)

To analyze the performance of our Diff-CIE under different diffusion settings, we follow previous works [16, 17] to explore two key parameters: the sampling timestep l and the size of selected fact-condition pairs K . We chose different combinations of l and K and analyze their impact on model effectiveness and efficiency, as shown in Figure 4.

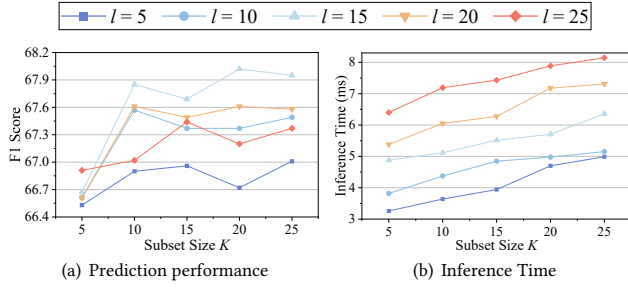


Figure 4: Performance with different subset size of fact-condition pairs K and different number of sampling steps l on Hyper-Temporal.

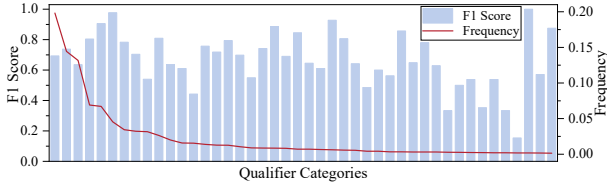


Figure 5: F1 score for each conditional qualifier on HyperRED dataset. The predicate categories are ordered by the number of samples in the training set.

Overall, moderately increasing l and K has a positive effect on model performance. Specifically, when K increases from 5 to 10, the model performance improves rapidly, likely because a smaller K may not cover all potential conditional instances. As K continues to increase, the performance shows a fluctuating upward trend, indicating diminishing marginal returns. The sampling steps l achieves better performance in the medium range. When l increases from 5 to 15, the model generally exhibits a steady improvement in most cases. However, when an excessively large l (e.g., $l = 25$) leads to saturated performance and even slightly declines.

For model efficiency, larger K and longer l both increase the inference burden. Comparatively, the sampling steps have a more pronounced influence on computational cost. when l increases from 5 to 25, the average inference time nearly doubles, whereas the overhead from increasing the size of selected pairs is relatively mild. Therefore, considering the trade-off between performance and efficiency, we adopt $l = 10$ and $K = 10$ as our default settings.

5.7 In-depth Analysis (RQ5)

5.7.1 Conditional Qualifier Performance Distribution. To further analyze the model’s performance distribution across different conditional qualifiers, we evaluate the per-category extraction F1 score of conditional qualifiers, as shown in Figure 5.

We can observe that despite significant variations in occurrence frequency across categories, the F1 scores of our Diff-CIE are relatively evenly distributed. It maintains good performance even for low-frequency categories, showing no significant decline as category frequency decreases. This is attributed to the reduced interaction space, which alleviates the learning difficulty and enhances prediction balance, indicating the robustness and generalization of our method when handling categories with varying frequencies.

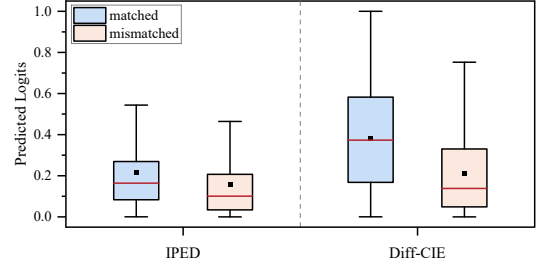


Figure 6: Predicted logits distribution of IPED and Diff-CIE on ComplexTRED.

5.7.2 Predicted Logits Distribution. We analyze the logits distribution for quintuple predictions to understand the model’s prediction mechanism and assess its fine-grained prediction reliability. Based on whether the predictions match the ground truth, we divide the results of IPED and Diff-CIE into matched and mismatched groups. Figure 6 visualizes the processed predicted logits.

As can be seen, the predicted logits of IPED are mostly concentrated in the lower range, with matched logits only slightly higher than mismatched ones, leading to substantial overlap and weak separability. In contrast, Diff-CIE exhibits a wider score distribution with a clearer separation trend. Moreover, the score distribution of matched samples in our method shifts upward overall, reflecting its inherent tendency and increased confidence in correct predictions.

This can be attributed the hierarchical node matching mechanism, which reduces the interference of irrelevant information and enables Diff-CIE to focus on more informative fact and condition nodes. Therefore, our method can estimate prediction confidence more effectively, making its outputs more reliable. Additionally, the larger score gap between the two groups allows for flexible threshold setting, enabling fine-grained threshold refinement to balance precision and recall as needed.

6 Conclusion

In this paper, we present a diffusion model on fact-condition star graph (Diff-CIE) for effective condition information extraction, facilitating the high-reliability KG construction for the semantic web. We convert conditional quintuples as a star graph, explicitly modeling the hierarchical dependencies between fact triples and conditional qualifiers. Building on a diffusion model, we then reformulate the CIE task as a denoising process progressively refining a fixed set of noised proposal nodes. In addition, a hybrid matching strategy introduces in-order matching to provide stable guidance. Extensive experiments demonstrate that our method consistently outperforms previous state-of-the-art methods.

Despite promising results, our method has several limitations. First, it increases computational overhead due to additional parameters. Second, the fixed star graph may be less effective in handling complex, multi-level dependencies. Future work will focus on developing lightweight architectures and flexible graph structures to enhance efficiency and adaptability.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (No.U23B2057, No.72071145).

References

- [1] Tianwen Jiang, Tong Zhao, Bing Qin, Ting Liu, Nitesh Chawla, and Meng Jiang. 2019. Multi-Input Multi-Output Sequence Labeling for Joint Extraction of Fact and Condition Tuples from Scientific Text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 302–312.
- [2] Tianwen Jiang, Tong Zhao, Bing Qin, Ting Liu, Nitesh V. Chawla, and Meng Jiang. 2019. The Role of Condition: A Novel Scientific Knowledge Graph Representation and Construction Model. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1634–1642.
- [3] Weijian Yu, Jie Yang, and Dingqi Yang. 2024. Robust Link Prediction over Noisy Hyper-Relational Knowledge Graphs via Active Learning. In *Proceedings of the ACM Web Conference 2024*. 2282–2293.
- [4] Yunxiao Yang, Jianting Chen, and Yang Xiang. 2024. A review on the reliability of knowledge graph: from a knowledge representation learning perspective. *World Wide Web* 28, 1 (2024), 4.
- [5] Yew Ken Chia, Lidong Bing, Sharifah Mahani Aljunied, Luo Si, and Soujanya Poria. 2022. A Dataset for Hyper-Relational Extraction and a Cube-Filling Approach. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 10114–10133.
- [6] Paolo Rosso, Dingqi Yang, and Philippe Cudré-Mauroux. 2020. Beyond Triplets: Hyper-Relational Knowledge Graph Embedding for Link Prediction. In *Proceedings of the Web Conference 2020*. 1885–1896.
- [7] Haiyun Jiang, Qiaoben Bao, Qiao Cheng, Deqing Yang, Li Wang, and Yanghua Xiao. 2020. Complex Relation Extraction: Challenges and Opportunities. *arXiv preprint arXiv:2012.04821* (2020).
- [8] Yu Liu, Quanming Yao, and Yong Li. 2021. Role-Aware Modeling for N-ary Relational Knowledge Bases. In *Proceedings of the Web Conference 2021*. 2660–2671.
- [9] Zhao Li, Xin Wang, Jun Zhao, Feng Feng, Zirui Chen, and Jianxin Li. 2025. HySAE: An Efficient Semantic-Enhanced Representation Learning Model for Knowledge Hypergraph Link Prediction. In *Proceedings of the ACM on Web Conference 2025*. 86–97.
- [10] Jiecheng Li, Xudong Luo, Guangquan Lu, and Shichao Zhang. 2025. Hyper-Relational Knowledge Representation Learning with Multi-Hypergraph Disentanglement. In *Proceedings of the ACM on Web Conference 2025*. 3288–3299.
- [11] Yaojia Lv, Zihao Zheng, Ming Liu, and Bing Qin. 2023. Conditional Knowledge Graph: Design, Dataset and a Preliminary Model. In *Proceedings of the 17th China Conference on Knowledge Graph and Semantic Computing*. Haofen Wang, Xianpei Han, Ming Liu, Gong Cheng, Yongbin Liu, and Ningyu Zhang (Eds.). 207–219.
- [12] Robin Jia, Cliff Wong, and Hoifung Poon. 2019. Document-Level N-ary Relation Extraction with Multiscale Representation Learning. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 3693–3704.
- [13] Kosuke Akimoto, Takuya Hiraoka, Kunihiro Sadamasa, and Mathias Niepert. 2019. Cross-Sentence N-ary Relation Extraction using Lower-Arity Universal Schemas. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 6225–6231.
- [14] Tong Zhu, Junfei Ren, Zijian Yu, Mengsong Wu, Guoliang Zhang, Xiaoye Qu, Wenliang Chen, Zhefeng Wang, Baoxing Huai, and Min Zhang. 2023. Mirror: A Universal Framework for Various Information Extraction Tasks. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 8861–8876.
- [15] Daniel Hernández, Aidan Hogan, and Markus Krötzsch. 2015. Reifying RDF: What Works Well With Wikidata?. In *Proceedings of the 11th International Workshop on Scalable Semantic Web Knowledge Base Systems*. 32–47.
- [16] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. DiffusionNER: Boundary Diffusion for Named Entity Recognition. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. 3875–3890.
- [17] Jianli Zhao, Changhao Xu, and Bin. Jiang. 2024. IPED: An Implicit Perspective for Relational Triple Extraction based on Diffusion Model. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2080–2092.
- [18] Shoufa Chen, Peize Sun, Yibing Song, and Ping Luo. 2023. DiffusionDet: Diffusion Model for Object Detection. In *Proceedings of the 2023 IEEE/CVF International Conference on Computer Vision*. 19773–19786.
- [19] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. 2020. End-to-End Object Detection with Transformers. In *Proceedings of the 16th European Conference on Computer Vision*, Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm (Eds.). 213–229.
- [20] Feng Li, Hao Zhang, Shilong Liu, Jian Guo, Lionel M. Ni, and Lei Zhang. 2024. DN-DETR: Accelerate DETR Training by Introducing Query DeNoising. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 4 (2024), 2239–2251.
- [21] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, and Heung-Yeung Shum. 2023. DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection. In *Proceedings of the 11th International Conference on Learning Representations*.
- [22] Xiuquan Hou, Meiqin Liu, Senlin Zhang, Ping Wei, Badong Chen, and Xuguang Lan. 2025. Relation DETR: Exploring Explicit Position Relation Prior for Object Detection. In *Proceedings of the 20th European Conference on Computer Vision*, Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (Eds.). 89–105.
- [23] Jiahao Wang and Jing Yang. 2023. A Three-Stage Pipeline for Conditional Entity and Relation Extraction. In *Proceedings of the 2023 International Joint Conference on Neural Networks*. 1–8.
- [24] Pengrui Ren, Tianyu Xu, Jianfeng Qu, Yu Sang, Zhixu Li, Junhua Fang, Pengpeng Zhao, and Guilin Ma. 2023. Tuning N-ary relation extraction as Machine Reading Comprehension. *Neurocomputing* 562 (2023), 126893.
- [25] Chenhan Yuan, Ryan Rossi, Andrew Katz, and Hoda Eldardiry. 2025. A Reinforcement Learning Framework for N-Ary Document-Level Relation Extraction. *IEEE Transactions on Big Data* 11, 2 (2025), 512–523.
- [26] Yuchen Zhuang, Yinghao Li, Junyang Zhang, Yue Yu, Yingjun Mou, Xiang Chen, Le Song, and Chao Zhang. 2022. ReSel: N-ary Relation Extraction from Scientific Text and Tables by Learning to Retrieve and Select. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 730–744.
- [27] Nanyun Peng, Hoifung Poon, Chris Quirk, Kristina Toutanova, and Wen-tau Yih. 2017. Cross-Sentence N-ary Relation Extraction with Graph LSTMs. *Transactions of the Association for Computational Linguistics* 5 (2017), 101–115.
- [28] Linfeng Song, Yue Zhang, Zhiguo Wang, and Daniel Gildea. 2018. N-ary Relation Extraction using Graph-State LSTM. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2226–2235.
- [29] Sanghak Lee, Seungmin Seo, Byungkook Oh, Kyong-Ho Lee, Donghoon Shin, and Yeonsoo Lee. 2020. Cross-sentence N-ary Relation Extraction using Entity Link and Discourse Relation. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*. 705–714.
- [30] Xiaojun Xue, Chunxia Zhang, Tianxiang Xu, and Zhendong Niu. 2023. Constrained Tuple Extraction with Interaction-Aware Network. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. 11430–11444.
- [31] Haoran Luo, Haihong E, Yuhao Yang, Tianyu Yao, Yikai Guo, Zichen Tang, Wentai Zhang, Shiyao Peng, Kaiyang Wan, Meina Song, Wei Lin, Yifan Zhu, and Anh Tuan Luu. 2024. Text2NKG: Fine-Grained N-ary Relation Extraction for N-ary relational Knowledge Graph Construction. In *Proceedings of the 38th Annual Conference on Neural Information Processing Systems*.
- [32] Bowen Yu, Yucheng Wang, Tingwen Liu, Hongsong Zhu, Limin Sun, and Bin Wang. 2021. Maximal Clique Based Non-Autoregressive Open Information Extraction. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 9696–9706.
- [33] Lu Yang, Jiajia Li, En Ci, Lefei Zhang, Zuchao Li, and Ping Wang. 2025. Label Drop for Multi-Aspect Relation Modeling in Universal Information Extraction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*. 5021–5040.
- [34] Patrick Ernst, Amy Siu, and Gerhard Weikum. 2018. HighLife: Higher-arity Fact Harvesting. In *Proceedings of the 2018 World Wide Web Conference*. 1013–1022.
- [35] Seungmin Seo, Byungkook Oh, Jeongeom Jeoung, Donghyun Kim, Kyong-Ho Lee, Dong-Hoon Shin, and Yeonsoo Lee. 2023. Active learning for cross-sentence n-ary relation extraction. *Information Sciences* 645 (2023), 119328.
- [36] Jiahao Chen, Haoyuan Ouyang, Junyang Ren, Wentao Ding, Wei Hu, and Yuzhong Qu. 2024. Timeline-based Sentence Decomposition with In Context Learning for Temporal Fact Extraction. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 3415–3432.
- [37] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. In *Proceedings of the 34th Annual Conference on Neural Information Processing Systems*. 6840–6851.
- [38] Jiaming Song, Chenlin Meng, and Stefano Ermon. 2021. Denoising Diffusion Implicit Models. In *Proceedings of the 9th International Conference on Learning Representations*.
- [39] Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, and Tatsunori B. Hashimoto. 2022. Diffusion-LM Improves Controllable Text Generation. In *Proceedings of the 36th Annual Conference on Neural Information Processing Systems*.
- [40] Emiel Hoogeboom, Didrik Nielsen, Priyank Jaini, Patrick Forré, and Max Welling. 2021. Argmax Flows and Multinomial Diffusion: Learning Categorical Distributions. In *Proceedings of the 35th Annual Conference on Neural Information Processing Systems*. 12454–12465.
- [41] Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and Lingpeng Kong. 2023. DiffuSeq: Sequence to Sequence Text Generation with Diffusion Models. In *Proceedings of the 11th International Conference on Learning Representations*.
- [42] Haopeng Zhang, Xiao Liu, and Jiawei Zhang. 2023. DiffuSum: Generation Enhanced Extractive Summarization with Diffusion. In *Findings of the Association for Computational Linguistics: ACL 2023*. 13089–13100.
- [43] Xiangyun Dong, Wei Li, Yuquan Le, Zhangyue Jiang, Junxi Zhong, and Zhong Wang. 2025. TermDiffuSum: A Term-guided Diffusion Model for Extractive Summarization of Legal Documents. In *Proceedings of the 31st International Conference*

- on *Computational Linguistics*. 3222–3235.
- [44] Lei Luo and Yajing Xu. 2023. Context-Aware Prompt for Generation-based Event Argument Extraction with Diffusion Models. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 1717–1725.
- [45] Xiao Long, Liansheng Zhuang, Aodi Li, Houqiang Li, and Shafei Wang. 2024. Fact Embedding through Diffusion Model for Knowledge Graph Completion. In *Proceedings of the ACM Web Conference 2024*. 2020–2029.
- [46] Oscar Sainz, Iker Garcia-Ferrero, Rodrigo Agerri, Oier Lopez de Lacalle, German Rigau, and Eneko Agirre. 2024. GoLLIE: Annotation Guidelines improve Zero-Shot Information-Extraction. In *Proceedings of the 12th International Conference on Learning Representations*.
- [47] Yaojie Lu, Qing Liu, Dai Dai, Xinyan Xiao, Hongyu Lin, Xianpei Han, Le Sun, and Hua Wu. 2022. Unified Structure Generation for Universal Information Extraction. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. 5755–5772.
- [48] Xiao Wang, Weikang Zhou, Can Zu, Han Xia, Tianze Chen, Yuansen Zhang, Rui Zheng, Junjie Ye, Qi Zhang, Tao Gui, Jihua Kang, Jingsheng Yang, Siyuan Li, and Chunsai Du. 2023. InstructUIE: Multi-task Instruction Tuning for Unified Information Extraction. *arXiv preprint arXiv:2304.08085* (2023).
- [49] Yaojie Lu, Hongyu Lin, Jin Xu, Xianpei Han, Jialong Tang, Annan Li, Le Sun, Meng Liao, and Shaoyi Chen. 2021. Text2Event: Controllable Sequence-to-Structure Generation for End-to-end Event Extraction. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*. 2795–2806.
- [50] Hao Fei, Shengqiong Wu, Jingye Li, Bobo Li, Fei Li, Libo Qin, Meishan Zhang, Min Zhang, and Tat-Seng Chua. 2022. LasUIE: Unifying Information Extraction with Latent Adaptive Structure-aware Generative Language Model. In *Proceedings of the 36th Annual Conference on Neural Information Processing Systems*. 15460–15475.

A Algorithm Procedure

We provide the training and inference procedure in Diff-CIE here.

Algorithm 1: Training Stage

Input : sentence s with a set of quintuples $\mathcal{F}$

- 1 **while** not convergence **do**
- 2 convert $\mathcal{F}$ to star graph $\mathcal{G}$
- 3 calculating the node attributes in $\mathcal{G}$ to get fact nodes $\mathcal{V}_f$ and condition nodes $\mathcal{V}_c$
- 4 node padding: $\chi_0 = (\chi_0^f, \chi_0^c) \leftarrow (\mathcal{V}_f, \mathcal{V}_c)$
- 5 $t \sim \text{Uniform}(\{1, \dots, T\})$, $\epsilon \sim \mathcal{N}(0, 1)$
- 6 $\chi_t = \sqrt{\alpha_t} \chi_0 + \sqrt{1 - \alpha_t} \epsilon$
- 7 calculate $\mathbf{p}_f^k, \mathbf{p}_c^k, \mathbf{p}_f^{\text{cls}}, \mathbf{p}_c^{\text{cls}}$ via $f_\theta(\chi_t, t, s)$
- 8 Take the gradient descent step on
 $\mathcal{L} = \mathcal{L}_{\text{permutation}} + \lambda \mathcal{L}_{\text{in-order}}$
- 9 **end while**

Algorithm 2: Inference Stage

Input : sentence s

Output : predicted star graph $\hat{\mathcal{G}}$

- 1 $\chi_T \sim \mathcal{N}(0, 1)$
- 2 z : an increasing sub-sequence of length l , $z_l = T$
- 3 **for** $i = l, \dots, 1$ **do**
- 4 calculate $\mathbf{p}_f^k, \mathbf{p}_c^k, \mathbf{p}_f^{\text{cls}}, \mathbf{p}_c^{\text{cls}}$ via $f_\theta(\chi_{z_i}, z_i, s)$
- 5 calculate $\hat{\epsilon}_{z_i}$ from $\mathbf{p}_f^k, \mathbf{p}_c^k, \mathbf{p}_f^{\text{cls}}, \mathbf{p}_c^{\text{cls}}$
- 6 $\hat{\chi}_{z_{i-1}} = \sqrt{\alpha_{z_{i-1}}} \hat{\chi}_0 + \sqrt{1 - \alpha_{z_{i-1}}} \hat{\epsilon}_{z_i}$
- 7 **end for**
- 8 filter $\hat{\chi}_0$ with threshold γ
- 9 reconstruct nodes into the final star graph $\hat{\mathcal{G}}$
- 10 **return** $\hat{\mathcal{G}}$

B Dataset Description

HyperRED is a general, large-scale dataset that covers diverse domains and multiple condition types. Hyper-Temporal is a subset of HyperRED which focuses on temporal conditional information and excludes relations with little correlation with time. ComplexTRED focuses on temporal conditions in complex scenarios. The statistics of the three datasets are shown in Table 5.

Table 5: Statistics of datasets.

Dataset	# Train	# Dev	# Test	# quintuple	$ \mathcal{R} $	$ \mathcal{Q} $
HyperRED	39,840	1,000	4,000	54,677	62	44
Hyper-Temporal	17,004	432	1,712	26,480	45	4
ComplexTRED	16,573	1,679	1,584	41,619	40	4

C Evaluation Protocol

To ensure a fair comparison with previous works, we report micro Precision, Recall, and F1-score. For the predicted conditional quintuple, the entire quintuple which completely matches the ground truth can be considered correct.

D Implementation Details

We adopt DeBERTa as the backbone of our method. We employ the AdamW optimizer with a linear decay scheduler for optimization, the initial learning rate is set as $5e-5$. For the diffusion model, the total timestep T is set to 1000, the sampling timestep l is 10, the number of selected node pairs K is 10. The filtering threshold γ of the prediction results is 6.5. All experiments are conducted on two NVIDIA 6000 Ada GPUs.

E Training Process Analysis

To intuitively demonstrate the learning dynamics of our model, Figure 7 illustrates the changes in the number of predictions and evaluation metrics on the dev set during training.

Overall, the number of predictions rapidly increases from 0 and eventually stabilizes around 40 epochs without significant fluctuations. Specifically, in the early stages (0-10 epochs), the model is relatively conservative, with the number of predictions far below the ground truth, but it maintains a high precision. As training progresses, both the number of predictions and correct predictions gradually increase, while recall exhibits a steady upward trend.

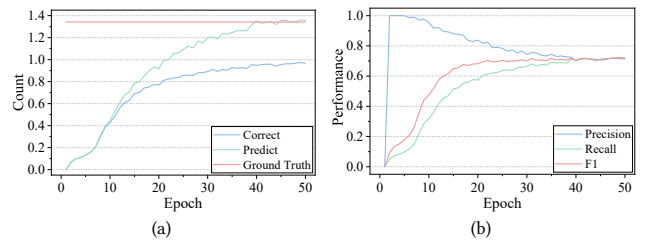


Figure 7: (a) The changes in the number of ground truths, predictions and accurate predictions during training. (b) Changes in precision, recall, and F1 on the dev set.